Contents

Syllabus **1**
ENPM663 Agentic Physical AI for Manufacturing Robotics . . . 1
Course Description . . . 2
Learning Outcomes . . . 2
Required Resources . . . 3
Prerequisites . . . 3
Course Structure . . . 3
Team Formation . . . 4
Major Assignments . . . 4
Final Project Descriptions . . . 4
Grading Structure . . . 6
Peer-Review Contribution (15% of grade) . . . 7
Academic Integrity . . . 8
Tips for Success in this Course . . . 9
Policies and Resources for Graduate Courses . . . 9
Names, Pronouns, and Self-Identifications . . . 10
Communication with Instructor . . . 10
Course Outline . . . 10
Resources and Accommodations . . . 11
Participation . . . 12
Course Evaluation . . . 13
Copyright Notice . . . 13

Syllabus

ENPM663 Agentic Physical AI for Manufacturing Robotics

Spring 2027

Field	Value	Field	Value
Section	0101 & RO01	**Professor**	Z. Kootbally
Credits	3	**Pronouns**	He/Him
Course Dates	Jan 27 to May 5, 2027	**Email**	zeidk@umd.edu
Course Times	Wednesdays, 7:00 to 9:40 pm	**Office Hours**	By appointment
Classroom	0101: TBA; RO01: online (details on ELMS)		

Course Description

This course teaches how to build agentic physical AI: robot systems in which AI agents decide what a factory cell does next and robot skills carry out each decision. An agent is a program that runs a loop around a large language model (LLM): it observes the state of the world, asks the model what to do, calls a tool (here, a robot skill), checks the result, and repeats.

The course is built on the Agile Robotics for Industrial Automation Competition (ARIAC) 2025 (https://pages.nist.gov/ARIAC_docs/en/latest/), a NIST simulation of an EV battery factory in which robots inspect battery cells, build kits and assemble modules. Students start from a classical ROS 2 stack (behavior trees, perception, PDDL planning) and then add AI: LLM agents that plan with tool calls and structured output, vision-language models (VLMs) that describe what the cameras see, and replanning when the factory does not behave as planned. Motion planning and grasping stay classical (MoveIt 2). The course closes with vision-language-action (VLA) models, robot foundation models and world models.

Students entering this course are expected to have working knowledge of ROS 2 (publishers and subscribers, services, actions, parameters, launch files) and Python. Week 1 provides a refresher on the ROS 2 topics most directly used in the course (QoS, executors, actions, TF2).

All project work is team-based. See the Team Formation and Peer-Review Contribution sections for details.

Learning Outcomes

After successfully completing this course you will be able to:

- Design agentic robot architectures that combine classical components (perception, planning, control) with AI components (LLMs, VLMs), and say which layer each one belongs to.
- Build symbolic models of an ARIAC scene and its tasks (frames, ontologies, hierarchical world models) that both a PDDL planner and an LLM agent can use.
- Implement behavior trees in `py_trees_ros` that run robot skills and handle failures.
- Generate executable plans for a robot cell with a classical PDDL planner.
- Build LLM agents that plan in an observe, act, observe loop, return structured output checked against a JSON schema, and call robot skills as tools through the Model Context Protocol (MCP).
- Use vision-language models to ground an agent's decisions in camera images and to catch perception failures.
- Design recovery and replanning for systems where LLM and VLM outputs can be wrong.
- Plan and execute pick and place motions with MoveIt 2.
- Compare the modular stack built in this course with vision-language-action models and robot foundation models, and use a hosted embodied-reasoning model for one perception step.
- Evaluate agentic robot systems with metrics for plan validity, grounding accuracy, execution success and end-to-end task performance.

Required Resources

- **Course Website:** elms.umd.edu
- **Operating system:** Ubuntu 24.04.x LTS (Noble Numbat): https://releases.ubuntu.com/noble/
- **Middleware:** ROS 2 Jazzy Jalisco: https://docs.ros.org/en/jazzy/Installation.html
- **Simulator:** Gazebo Harmonic: https://gazebosim.org/docs/harmonic/
- **Editor:** Visual Studio Code: https://code.visualstudio.com/
- **ARIAC 2025:** https://pages.nist.gov/ARIAC_docs/en/latest/ (Docker or native install; the final demo runs from a Dockerfile)
- **AI access (from GP4 on):** Google Gemini by default. For development, your team uses its own free-tier Gemini API key. The course Gemini key runs only the graded GP4 and final demo evaluations. A team may run a local open-weight model instead, as the GP4 and final demo pages allow.

Prerequisites

Working knowledge of ROS 2 is required. Specifically, students should be comfortable with:

- Creating and managing ROS 2 packages and workspaces.
- Writing publishers, subscribers, services, and action clients and servers in Python.
- Using launch files, parameters, and basic command-line tools (`ros2 topic`, `ros2 node`, etc.).
- Reading C++ ROS 2 code (writing C++ is not a prerequisite).
- Using Git and GitHub well: cloning, branching, committing, pushing and pulling, resolving a merge conflict, and opening and reviewing a pull request. The course does not teach them.

A diagnostic quiz in Week 1 tests these prerequisites: ROS 2, and Git and GitHub. Students and the instructor see each score. The quiz is ungraded.

Course Structure

- The class meets once a week for a lecture with instructor demonstrations. There are no in-class exercises. Before each class, review the previous lectures. Early in the course, expect roughly 4 hours per week on group project work. As the course progresses, projects may require 6 to 8 hours per week per student.
- Quizzes are given in class (unless you are registered in the online section). Absent students cannot take quizzes.
- Class attendance and active participation are not graded but are encouraged.
- Late submissions are penalized 10% of the project's grade per calendar day late, up to 3 days; submissions more than 3 days late receive no credit. Extensions are granted only when more than one team needs additional time on a submission.
- No quiz or group project is dropped. The final demo is required; nothing replaces its grade.
- The final demo is graded on an ARIAC trial your team has not seen. The code freeze is Friday, April 30, 2027, at 11:59 pm. Before the Week 15 class, the instructor runs every team's Docker image, built from its code-freeze tag `final-demo`, on that trial on one machine, so every team

gets the same hardware. In class, each team presents its approach, design decisions and results, with its recorded run.
- All projects use a peer-review system. Each team member rates the contributions of the other team members after each group project and after the final demo. See Peer-Review Contribution.

Team Formation

- All project work is team-based: four group projects (GP1 to GP4) and the final demo.
- **Team size: 3 students.** Teams of 2 are allowed only with instructor approval (e.g., when cohort numbers do not divide evenly or for remote students). Teams of 4 or more are not permitted; integration projects of this scope typically generate coordination overhead without productivity gain past three members.
- **Teams are formed in Week 2**, the day GP1 is posted. Each team posts a one-page team charter on ELMS before Week 3: who owns which part of the system, when the team meets, how it uses git, what happens when someone goes quiet, and how it will use AI tools.
- Teams stay together for all four group projects and the final demo. Stable team composition lets the peer-review signal accumulate across the semester.
- A team change request requires a written rationale and instructor approval, and is expected only in cases of sustained imbalance documented in peer review.

Major Assignments

- **Final Project.** Four cumulative group projects (GP1 to GP4) and a final demo that build an agentic system for the ARIAC 2025 factory. The final demo is graded on an unseen ARIAC trial, run by the instructor from your team's Docker image. Detailed instructions and submission guidelines are provided for each project. See the Course Outline for the schedule.
- **Quizzes.** Quizzes are given at the beginning of class, either on Canvas or on paper, and last 20 minutes. All quizzes are closed-notes.
- **Peer-Review Contribution.** After each group project and after the final demo, every student rates the contribution of their teammates. See Peer-Review Contribution.

Final Project Descriptions

The robot arms move through the **course skills package**: Python ROS 2 actions for picking and placing parts, built on MoveIt 2 and provided with GP1. Teams decide what the robots do; the skills package carries it out. ARIAC 2025 itself provides joint trajectory and gripper actions, but no pick or place skills. Teams move the AGVs with ARIAC's own AGV move action.

Write your behavior trees, the code that runs the planner and your LLM agent in Python. You may write your team's other nodes in C++, for example a perception node, if your Dockerfile builds them.

In ARIAC 2025, a cell on the inspection conveyor reaches the inspection robot only if your team has submitted a passing inspection report for it. A failing or missing report keeps the inspection door closed, and the cell drops into the inspection bin. So every project submits an inspection report for every cell.

GP1: Behavior Tree for One Kit

Posted: Week 2 / **Due:** Week 5 / **Weight:** 10%

Build a `py_trees_ros` behavior tree that builds one kit: feed cells onto the inspection conveyor, submit a passing inspection report for each cell, read their voltages, recycle the cells outside the voltage range, place four cells on an AGV tray, move the AGV to shipping and submit the kit. The leaves call the course skills package and the ARIAC services. No defect detection yet. GP1 comes with a starter tree: Weeks 2 and 3 go to setup, the skills package and the ARIAC services, and the tree work follows L4 (Week 4).

Deliverable: ROS 2 package with a launchable behavior tree, plus a short write-up (1 to 2 pages) with the tree diagram and the reason for each composite choice.

GP2: Cell Inspection

Posted: Week 5 / **Due:** Week 7 / **Weight:** 10%

Place lidar sensors around the inspection conveyor within the ARIAC sensor budget, as ARIAC 2025 requires, and detect dents, scratches and bulges from the point clouds. Submit an inspection report for each cell before it reaches the inspection door: a failing report sends the cell into the inspection bin. The GP1 tree then reads the voltage of each cell that passes and places or recycles it. Keep one record per cell that joins the two checks.

Deliverable: Perception package, the updated tree, and a results table with precision and recall per defect type and the sensor cost.

GP3: PDDL Planner and BT Executor

Posted: Week 7 / **Due:** Week 10 / **Weight:** 10%

Write a PDDL domain for Task 1 (inspection and kit building) with the two inspection robots and the AGVs, and execute its plans through a behavior tree built from the plan. Replan when a cell fails inspection. Choosing four cells that meet the kit voltage tolerance is a numeric problem, so it is a separate selection step. Spans spring break.

Deliverable: PDDL files, planner integration (Fast Downward or Unified Planning Framework), the executor, a demo on 2 to 3 trials (one of which triggers a replan), and a short reflection on what PDDL modeled well in Task 1 and what was hard to model.

GP4: LLM Agent with VLM Grounding

Posted: Week 10 / **Due:** Friday, April 23, at 11:59 pm (Week 13) / **Weight:** 10%

Replace the PDDL planner with an LLM agent that calls the robot skills as tools (through MCP if the team chooses) and returns structured output checked against a JSON schema. A VLM describes the cells for the agent. The same executor from GP3 runs its decisions. Test the agent on the situations GP3's domain does not model: a conveyor malfunction, a voltage tester malfunction and high-priority orders. (The vacuum tool malfunction affects only Task 2, so the final demo tests it.)

Deliverable: Agent module, VLM module, executor changes, a side-by-side comparison with GP3 on identical trials (validity, plan length, cost, latency), and a reflection on the agent's failure modes and how they were caught.

Final Demo

Posted: Week 12 / **Demo:** Week 15 / **Weight:** 20%

Integrate GP1 to GP4 into one system that runs a full ARIAC 2025 trial: Task 1 and Task 2 (module construction with the assembly robots and the gantry welder), with any of the ARIAC challenges. A hosted embodied-reasoning model (for example Gemini Robotics ER) judges cell defects from camera images over the inspection conveyor, and is scored with precision and recall on the same cells as the team's GP2 lidar detector. The trial used for grading is unseen: before the Week 15 class, the instructor runs every team's Docker image on it, on one machine.

Deliverable: Integrated codebase that runs from a Dockerfile (its code-freeze tag is what the instructor runs and grades), and a short presentation in class (5 to 6 minutes) with the recorded run, covering architecture, design decisions, failure modes encountered, and how they were handled, followed by 3 minutes of questions, one for each team member.

Grading Structure

Component	Percentage
Final Project (GP1 to GP4 and the final demo)	60%
Quizzes	25%
Peer-Review Contribution	15%
Total	**100%**

Final project weights

Component	Weight	Notes
GP1	10%	
GP2	10%	
GP3	10%	
GP4	10%	

Component	Weight	Notes
Final demo	20%	Graded on the instructor's run of your Docker image on an unseen ARIAC trial; presented in Week 15.

There is no final exam.

Final Grade Cutoffs

	+		+		+		+	
	97.00%		87.00%		77.00%		67.00%	
A	94.00%	B	84.00%	C	74.00%	D	64.00%	F < 60.00%
	-		-		-		-	
	90.00%		80.00%		70.00%		60.00%	

Final letter grades are assigned based on the percentage of total assessment points earned. Standards are applied consistently: being close to a cutoff is not the same as making the cut (89.99 is not 90.00).

Peer-Review Contribution (15% of grade)

The Peer-Review Contribution category is assessed *by teammates*, not by the instructor, and reflects intra-team contribution. It is separate from class participation, which is ungraded.

Instrument

After each of the four group projects and after the final demo, every student completes a peer-review form via ELMS-Canvas, rating each teammate (not themselves) on the following five dimensions:

1. **Technical contribution.** Did this teammate contribute substantively to the technical work (code, design decisions, debugging, system integration)?
2. **Reliability and communication.** Did this teammate show up to meetings, respond to messages, and deliver agreed work on time?
3. **Quality of work.** Was their contribution of a quality you could build on (clean, complete, tested code), or did you need to redo it?
4. **Knowledge and skill.** Did this teammate bring relevant knowledge or actively work to acquire what was needed?
5. **Team-player behavior.** Did this teammate help others, listen, accept feedback, and contribute to a constructive team environment?

Each dimension uses the following 5-point scale:

- **5 = Consistently outstanding.**
- **4 = Above expectations.**
- **3 = Meets expectations.**
- **2 = Below expectations; needs improvement.**
- **1 = Substantially below expectations; raises concern.**

A short written comment (one to three sentences) is required for each teammate. Comments are mandatory whenever a dimension is rated 2 or below, or 5. Uniform ratings (all 3s or all 5s for every teammate with no written justification) will be returned ungraded and must be redone.

Confidential concern flag

Each form has a confidential checkbox: "I want the instructor to follow up about team dynamics on this project." Checking it triggers a private check-in with the instructor. Teammates are not told the flag was raised.

How the score is computed

- For each project, a student's peer score is the mean of their teammates' ratings across all five dimensions, normalized to a 0 to 100 scale (score = $80 + 10 \times (\text{mean} - 3)$: a mean of 1 gives 60, 3 gives 80 and 5 gives 100; the scale rewards above-expectations contribution while protecting against single- teammate penalties).
- The project-level peer scores are averaged across all five (GP1 to GP4 and the final demo, equal weight) and multiplied by 15% to produce the final Peer-Review Contribution grade.
- Outliers are reviewed by the instructor before being applied. If one teammate scores a peer 1 across all dimensions while another scores them 4, the instructor reads the written comments and decides whether to include both ratings, discard the outlier, or initiate a team conversation.

Protections against gaming

- Ratings are visible to the instructor only, never to the rated student.
- A single low rating from one teammate cannot, by itself, push a student below 70 in this category; any team-averaged peer score under 70 triggers instructor review before being assigned.
- The instructor reserves the right to override peer-assigned scores in documented cases of reciprocal rating (you rate me high if I rate you high) or coordinated retaliation.

Academic Integrity

Some submissions will be collected via Turnitin on the course ELMS page to support scholarly writing and to verify the integrity of student work.

The University Code of Academic Integrity is designed to ensure that the principles of academic honesty and integrity are upheld. The University of Maryland does not tolerate academic dishonesty. All acts of academic dishonesty will be dealt with in accordance with the provisions of this code.

Course assistance websites (such as CourseHero) and AI-generated content are not permitted as sources unless the instructor explicitly grants permission. Specifically:

- Using AI tools (including LLM APIs) is required in GP4 and the final demo. The team must design and debug the code that calls these tools. Submitting AI-generated code that the team cannot explain is an academic integrity violation.
- Teams may use AI tools to clarify concepts or look up syntax for any group project, but the submitted code and report must be the team's own work, and any team member must be able to explain any portion of it on request.
- For quizzes, no external tools or notes are permitted.

On each group project submission, every team member must sign the following pledge: *"I pledge on my honor that I have not given or received any unauthorized assistance on this group project."* On each quiz, the same pledge applies to the individual student.

Tips for Success in this Course

1. **Participate.** Engage with the lectures, ask questions, and discuss course content with your teammates and classmates. Robotic systems are integration problems; you will learn faster by talking through design decisions with peers.
2. **Manage your time.** The course moves quickly. Block out time in your schedule for group project work and include extra time for environment and dependency issues.
3. **Coordinate with your team.** Set a recurring meeting time. Use a shared repository from day one. Agree early on who owns which subsystem.
4. **Log in regularly.** Check ELMS-Canvas several times a week for announcements and discussion replies. Check daily near submission deadlines.
5. **Do not fall behind.** Each week builds on the previous one. If you feel you or your team are falling behind, contact the instructor early.
6. **Use ELMS notification settings.** Enable instant or daily notifications so you do not miss announcements.
7. **Ask for help.** For ELMS or technology issues, contact IT support. For course content, contact the instructor or classmates.

Policies and Resources for Graduate Courses

It is our shared responsibility to know and abide by the University of Maryland policies that apply to all courses, including:

- Academic integrity
- Student and instructor conduct
- Accessibility and accommodations
- Attendance and excused absences
- Grades and appeals
- Copyright and intellectual property

See https://gradschool.umd.edu/course-related-policies for the full set of graduate course-related policies.

Names, Pronouns, and Self-Identifications

The University of Maryland recognizes the importance of a diverse student body and is committed to fostering inclusive and equitable classroom environments. You are invited, if you wish, to tell the instructor how you want to be referred to in this class, both in terms of your name and pronouns. The pronouns someone uses are not necessarily indicative of their gender identity. Visit lgbtq.umd.edu to learn more.

Whether and how to disclose how you identify in terms of gender, race, class, sexuality, religion, ability, and other aspects of your identity is your choice and should be self-identified, not presumed or imposed.

Communication with Instructor

- **Email.** Please email zeidk@umd.edu for personal or academic concerns. Please do not email about questions that are easily answered by the syllabus or ELMS (e.g., due dates, point values). The instructor answers within 24 hours on weekdays.
- **ELMS.** Important announcements (including project changes and due-date changes) are posted on ELMS. Enable email and announcement notifications to avoid missing messages.
- **Communication with peers.** With a diversity of perspectives and experience, disagreement and debate are expected. Conduct yourself professionally and respectfully. This course is not a space for hate speech, harassment, or derogatory language. Alert the instructor immediately if you feel threatened, dismissed, or silenced.

Course Outline

Lectures meet on Wednesdays. Spring break (Mar 14 to 21) includes Wednesday Mar 17.

Week	Date	Lecture and topic	Deliverables
1	01/27	L1: Course Overview and ROS 2 Refresher	Diagnostic quiz (ungraded)
2	02/03	L2: System Architectures for Agentic Robotics	Teams formed; GP1 posted
3	02/10	L3: Knowledge Representation for Agentic Robotics	Team charter due
4	02/17	L4: Behavior Trees for Task Execution	Quiz 1

Week	Date	Lecture and topic	Deliverables
5	02/24	L5: Sensing and Perception for Robotic Systems	Quiz 2; GP1 due; GP2 posted
6	03/03	L6: Task-Level Planning with PDDL	
7	03/10	L7: Plan Execution Patterns with Behavior Trees	Quiz 3; GP2 due; GP3 posted
8	03/17	Spring break: no class	
9	03/24	L8: LLM Agents I: Agent Loops, Tool Use, and Structured Output	
10	03/31	L9: LLM Agents II: VLM Grounding and Agentic Replanning	Quiz 4; GP3 due; GP4 posted
11	04/07	L10: Frames, Transforms, and Motion Planning	
12	04/14	L11: Recovery, Robustness, and Failure Handling for Agentic Systems	Quiz 5; final demo specification posted
13	04/21	L12: Evaluation and Measurement Science for Agentic Systems	GP4 due Fri 04/23
14	04/28	L13: The Physical AI Frontier: VLAs, Robot Foundation Models, and World Models	
15	05/05	Final demo and presentations	Final demo

Note: This is a tentative schedule and subject to change as necessary. Monitor the course ELMS page for current deadlines. In the event of a prolonged university closing or extended absence, adjustments to the schedule, deadlines, and assignments will be made based on the duration of the closing and the specific dates missed.

Resources and Accommodations

Accessibility and Disability Services

The University of Maryland is committed to creating and maintaining a welcoming and inclusive educational environment. The Accessibility & Disability Service (ADS) provides reasonable accom-

modations to qualified individuals. ADS cannot assist retroactively, so it is generally best to request accommodations several weeks before the semester begins or as soon as a disability becomes known. Any student who needs accommodations should contact the instructor as soon as possible.

For assistance, contact ADS at 301-314-7682 or adsfrontdesk@umd.edu.

Student Resources and Services

UMD provides a wide range of student support resources. See https://studentsuccess.umd.edu/ for the full list, including the Writing Center, the Counseling Center, and academic support services.

Notice of Mandatory Reporting

As a faculty member, the instructor is designated as a Responsible University Employee and must report all disclosures of sexual assault, sexual harassment, interpersonal violence, and stalking to UMD's Title IX Coordinator.

For confidential support, contact CARE to Stop Violence at 301-741-3442 or the Counseling Center at 301-314-7651.

You may also contact UMD's Title IX Coordinator at 301-405-1142 or titleIXcoordinator@umd.edu.

Basic Needs Security

If you have difficulty affording groceries or accessing sufficient food, or lack a safe and stable place to live, please contact UMD's Division of Student Affairs for information about available resources. Let the instructor know if there is any way the course can help.

Veteran Resources

UMD provides additional support for student veterans through the Office of Veteran Student Life and the Counseling Center. Veterans and active duty military personnel with special circumstances (e.g., deployments, drill requirements, disabilities) are encouraged to communicate these to the instructor in advance.

Participation

Attendance is important to note-taking and your performance. Class discussion is a critical component of learning. Each student is expected to make substantive contributions to their team's work. Students with a legitimate reason to miss a session should communicate in advance with the instructor, except in case of emergency. Students who miss a session are responsible for learning what they missed.

Class attendance and class-discussion participation are not directly graded. Intra- team contribution is graded via the Peer-Review Contribution category.

Course Evaluation

Please submit a course evaluation through Student Feedback on Course Experiences at the end of the semester. All information submitted is confidential.

Copyright Notice

Course materials are copyrighted and may not be reproduced for anything other than personal use without written permission.